Lab#5



– Digital Electronics –

Using interrupts to program a NIOS V system

The purpose of this laboratory session is to learn how to build and program a simple NIOS V-based system

Contents:

1. Using interrupts in a NIOS V system
2. Do it yourself

Abbreviations and acronyms:

IC – Integrated Circuit
LED – Light Emitting Diode
MUX – Multiplexer

USE EXACTLY THE SAME I/O PINS SPECIFIED IN THIS DOCUMENT.

1 – Using interrupts in a NIOS V system

Follow the tutorial in the appendix. It explains you how to increase your NIOS V system with QSYS (Platform Designer) and how to run a simple interrupt based program.

PLEASE REFER TO THE LAB#0 FILE FOR DETAILS ON PROGRAMMING THE FPGA

2 – Do it yourself

a. **Create** a new Quartus project with the following ports, which correspond to names already defined in the pin assignment .csv file.
- CLOCK_50: in std_logic / input CLOCK_50
- KEY: in std_logic_vector(2 downto 0) / input [2:0] KEY
- HEX0 : out std_logic_vector(0 to 6) / output [0:6] HEX0
- HEX1 : out std_logic_vector(0 to 6) / output [0:6] HEX1
- HEX2 : out std_logic_vector(0 to 6) / output [0:6] HEX2
- HEX3 : out std_logic_vector(0 to 6) / output [0:6] HEX3
- HEX4 : out std_logic_vector(0 to 6) / output [0:6] HEX4
- HEX5 : out std_logic_vector(0 to 6) / output [0:6] HEX5

b. Stemming from the tutorial **create** with the QSYS (Platform Designer) Tool a NIOS V processor system with
- one NIOS V/m processor
- a 256kiB one chip memory
- one memory interface to the external SDRAM
- one 2-bit input parallel port
- six 7-bit output parallel ports
- one Timer Internal Peripheral
- a Performance Counter Internal Peripheral
- a JTAG UART

c. **Declare** the processor as a component inside your system and **instantiate** it connecting:
- the processor clock to CLOCK_50 and the reset to KEY(0).
- the 2 bit input port to KEY(2) and KEY(1) respectively.
- the 7 bit output ports to HEX0, … HEX5.

d. **Compile** the hardware system and verify correctness. Then download it on the DE1-SoC board.

e. **Write, compile and execute** a program doing the following tasks using interrupts:
- Initialize all the hardware.
- Compute and display on terminal the first 15 successive approximations of the mathematical constant *e* (i.e. Euler's Number) calculated as:

$$e_i = \sum_{n=0}^{i} \frac{1}{n!}$$

  Display each iteration with 10 decimal digits, measure its execution time and display it.
- Display on the 7-segment displays the word “dIGELn” rotating to the right if KEY(2) has been pressed and rotating to the left if KEY(1) has been pressed. If no key has been pressed the word is shown but not moving (‘P’-> HEX5, ‘O’ -> HEX4, … ‘t’ -> HEX1 and ‘O’-> HEX0). The moving interval must be half a second.

Suggestion:

- Build the hardware.
- With *Ashling RiscFree IDE for Intel FPGAs* create the main program, compile, execute and debug it without Interrupt Routines.
- Insert the Timer initialization and interrupt routine and verify the operation.
- Insert the KEY handling interrupt routine and complete the debug.

Deliverable report must contain:

- A snapshot of the final configuration of the QSYS (Platform Designer) tool.
- The top entity HDL description.
- The listings of the programs, with comments to explain the code.
- A snapshot of the *Ashling RiscFree IDE for Intel FPGAs* tool after one of the programs has been executed.

APPENDIX – Adding peripherals to a NIOS V-based system with QSYS (Platform Designer) and writing interrupt-based code.

READ THIS VERY CAREFULLY

1. Introduction

This tutorial is intended for creating mid-complexity systems based on NIOS V with QSYS (Platform Designer) and programming them using interrupts.

First, this tutorial explains how the SDRAM chip on Intel's DE1-SoC Development and Education board can be used with a Nios V system implemented by using the Intel QSYS (Platform Designer) tool.

The tutorial used in the last lab introduces the Intel QSYS (Platform Designer) System Integration Tool and explains how the memory in a Cyclone series FPGA chip can be used in the context of a simple Nios V system. For practical applications it is necessary to have a much larger memory. The Intel DE1-SoC board contains an SDRAM chip that can store 64 Mbytes of data. The memory is organized as 8M x 16 bits x 4 banks. The SDRAM chip requires careful timing control. To provide access to the SDRAM chip, the QSYS (Platform Designer) tool implements an SDRAM Controller circuit. This circuit generates the signals needed to deal with the SDRAM chip.

In the following, we will add several peripherals, including the SDRAM, to the Nios V system described in the previous lab.

The system implements the following functions:

1. The main processing task will be the computation of the first 12 factorial numbers. The task will be repeated 5000 times and at the end the 12th factorial will be printed together with the execution time of the task.
2. A counting task performed under interrupt which will update, with the least significant bits, the hexadecimal display HEX0. Counting will be performed at 1 second or at a 1/5 of a second intervals if either pushbutton KEY(2) or pushbutton KEY(1) is pressed. If no button has already been pressed no counting takes place.
3. Pushbutton KEY(0) is the processor reset.

2. The SDRAM interface

The SDRAM chip on the DE1-SoC board has a capacity of 512 Mbits (64 Mbytes). Each chip is organized as 8M x16 bits x 4 banks. The signals needed to communicate with the chip are shown in Figure 1. All of the signals, except the clock, can be provided by the SDRAM Controller that can be generated by using the QSYS (Platform Designer) tool. The clock signal is provided separately. It has to meet the clock-skew requirements as explained later in this tutorial. Note that some signals are active low, which is denoted by the suffix N.

Our starting point will be the Nios V system discussed in the previous lab tutorial, which we implemented in a project called *lights*.

If you saved the *lights* project, then open this project in the Quartus Prime software and then open the QSYS (Platform Designer) tool. Otherwise, you need to create and implement the project, as explained in the previous lab tutorial.

To add the SDRAM controller, in the window of Figure 2 we first have to add it as a QSYS (Platform Designer) component. So first download the *altera_avalon_new_sdram_controller.zip* file from the course material website and unzip it on the local PC. Then, from Platform Designer select **`Tools > Options`** and in the *IP Search Path* section click *Add* to add the path to the *altera_avalon_new_sdram_controller* folder. Now, from the IP catalog you can search for SDRAM and select SDRAM controller.

A window, depicted in Figure 3, appears.

1. Set the *Data Width* parameter to 16 bits, the *Address Width: Row* to 13 bits, the *Address Width: Column* to 10 bits, and leave the default values for the rest. Since we will not simulate the system in this tutorial, do not select the option *Include a functional memory model in the system testbench*.

2. Select the *Timing* tab to get to the window in Figure 4. Configure the SDRAM timing parameters by setting
   a. *Issue one refresh command every* to 7.8125 microseconds,
   b. *Duration of precharge command (t_rp)* to 15.0 nanoseconds,
   c. *ACTIVE to READ or WRITE delay (t_rcd)* to 15.0 nanoseconds,
   d. *Access time (t_ac)* to 5.4 nanoseconds. Click **`Finish`**.

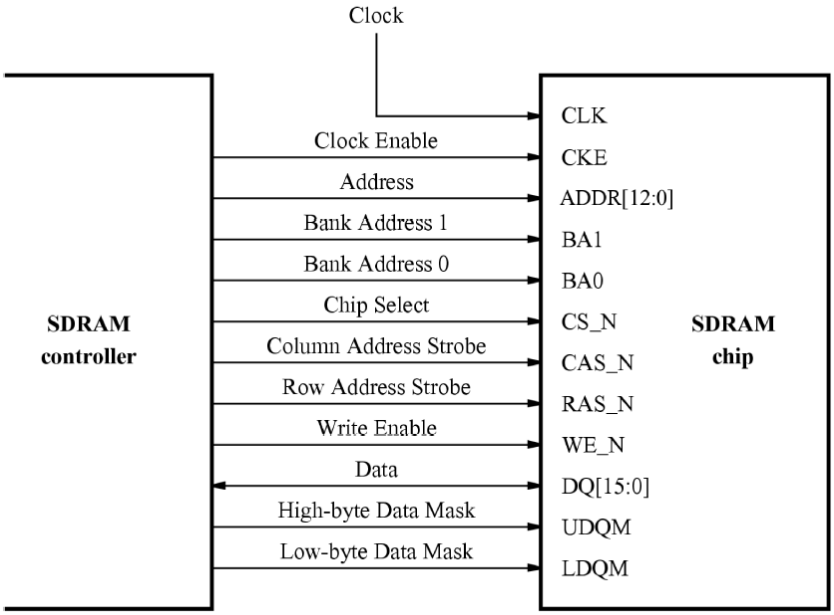


Figure 1 The SDRAM signals

Now, in the window of Figure 2, there will be an SDRAM controller module added to the design. Rename this module to *sdram*. Connect the SDRAM to the rest of the system in the same manner as the on-chip memory, and export the SDRAM wire port as *sdram_wire*.

The system obtained so far will not work correctly due to a clock skew on the DE1-SoC board. The clock skew depends on physical characteristics of the DE1-SoC board. For proper operation of the SDRAM chip, it is necessary that its clock signal, *DRAM_CLK*, leads the Nios V system clock, *CLOCK_50*, by 3 nanoseconds. This can be accomplished by using a phase-locked loop (PLL) circuit, which is created automatically using the Clock Signals IP core provided by the Intel FPGA University Program.

To add the Clock Signals IP core, in the QSYS (Platform Designer) tool, select **`University Program > Clock > System and SDRAM Clocks for DE-Series Boards`** and click **`Add`**. A window, depicted in Figure 5, appears. Select *DE1-SoC* from the *DE- Series Board* drop-down list. Click **`Finish.`**

Remove the system clock component *clk_0*. All other IP cores (including the SDRAM) should be adjusted to use the *sys_clk* output of the *Clock Signal core* as the clock and the *reset_source* output as the reset. Rename the *Clock Signal core* to *clocks* and export:

- the *sdram_clk* signal under the name *sdram_clk*,
- the *ref_clk* signal under the name *clk*,
- *ref_reset* signal under the name *reset*.

The system is shown in Figure 6.

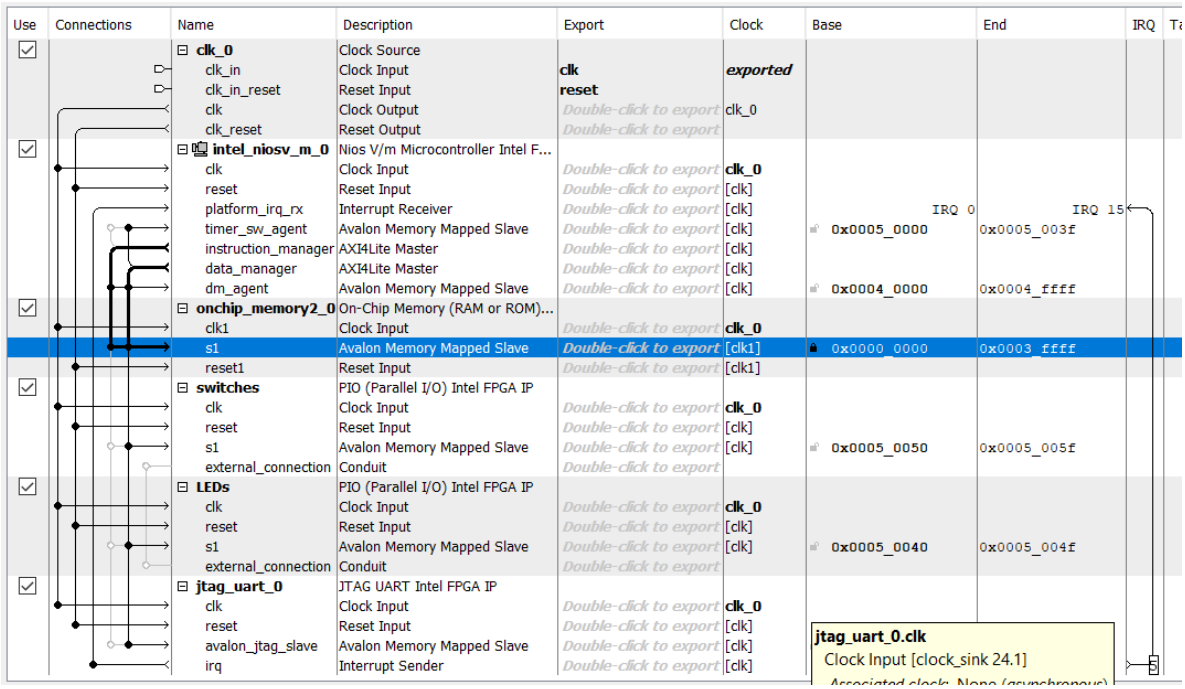


Figure 2 The Nios V system defined in the introductory tutorial

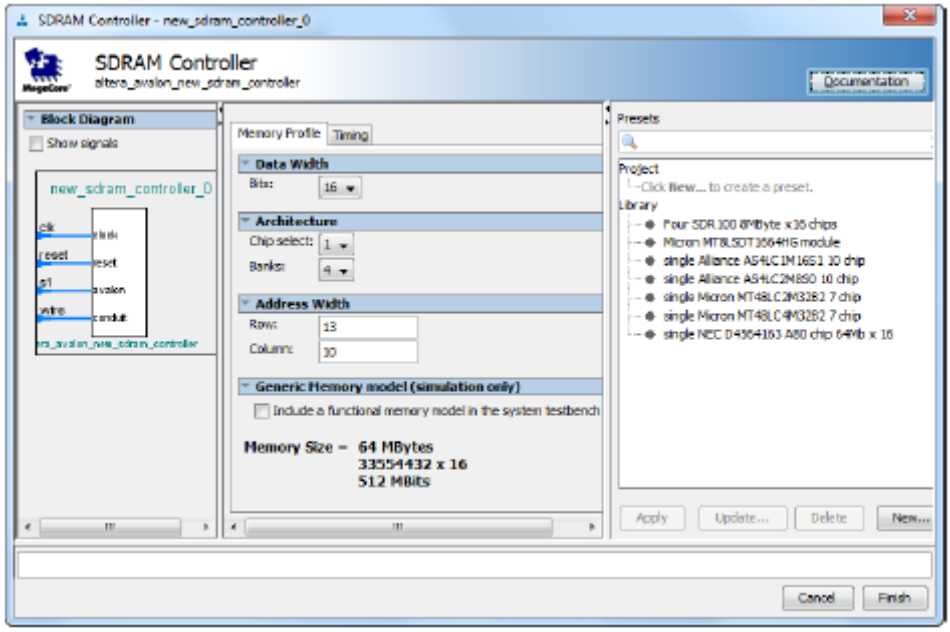


Figure 3 Add the SDRAM controller

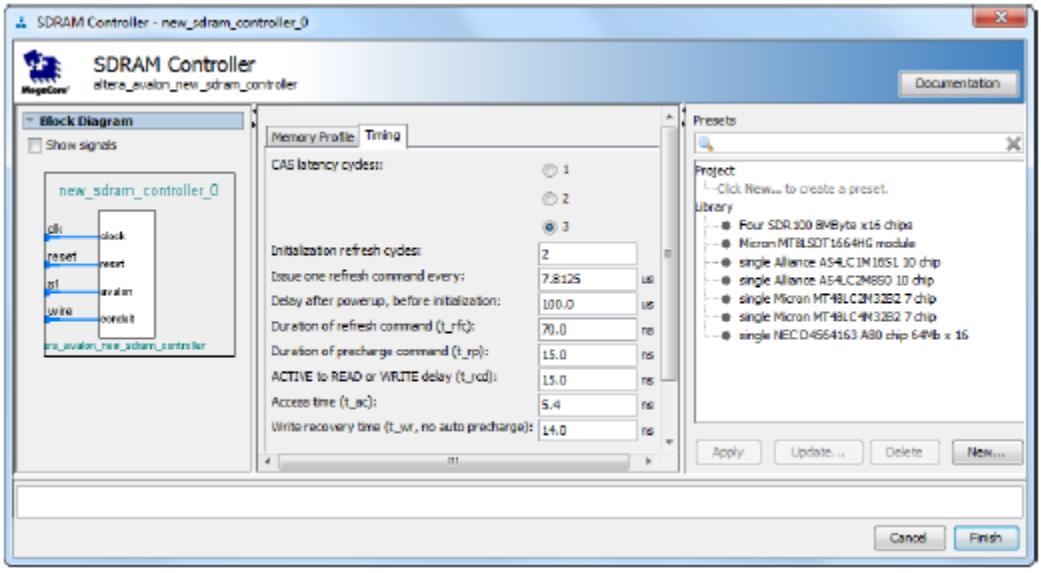


Figure 4 SDRAM timings

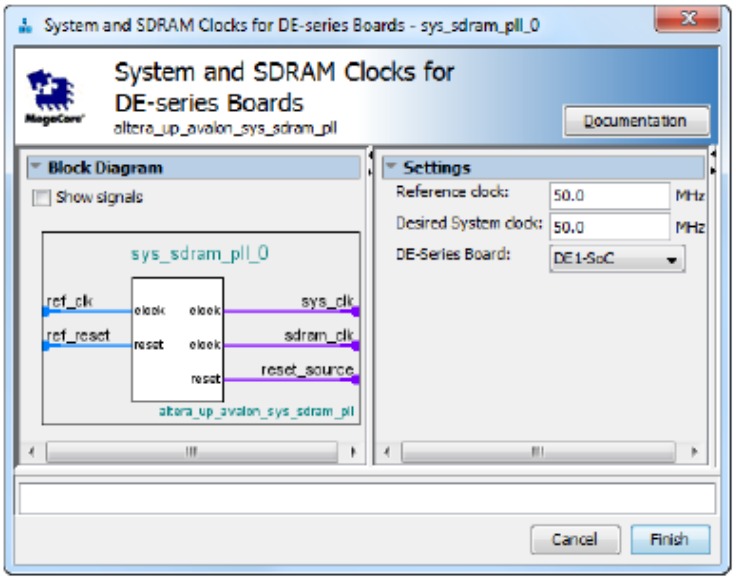


Figure 5 Clock signals IP core

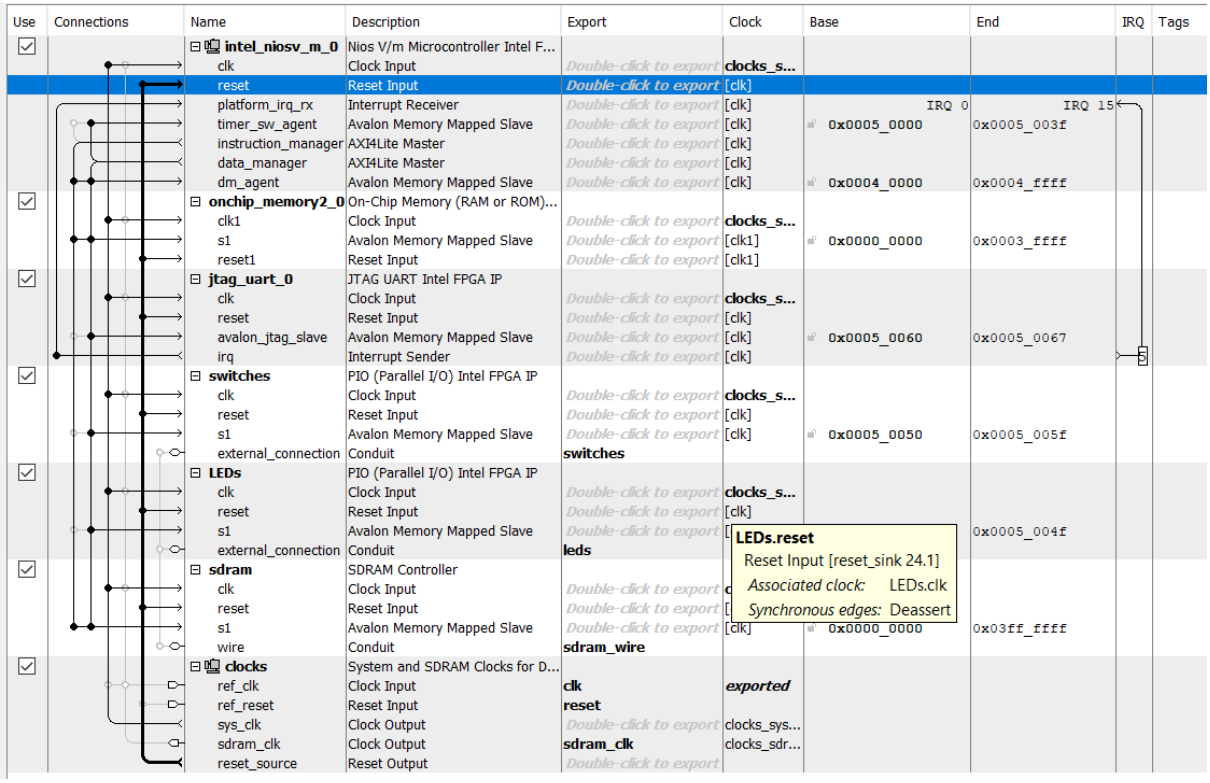


Figure 6 Nios V system with SDRAM controller and PLL

3. Modifying the PIOs

In order to obtain the system required for the application described above we have to modify the input and output PIOs.

Select the output PIO (*LEDs*) right click and select **`Edit`**. Modify the *width* to 7 and click **`Finish`**. Since this will be the port connected to the hexadecimal display (*HEX0*) it is useful to rename it: right click the PIO, select **`Rename`** and rename the port as *HEX0*. Rename also the *external connection* as *hex0*.

Select the input PIO (*switches*) right click and select **`Edit`**. Modify the *width* to 2 and enable interrupt generation as follows (see Figure 7).

- In the *Edge Capture Register* section choose:
  - *Synchronously capture*
  - *Edge type* **`FALLING`**
  - *Enable bit-clearing for edge capture register*
- In the *Interrupt* section choose:
  - *Generate IRQ*
  - *IRQ Type* **`EDGE`**

Then, click **`Finish.`** This makes the input PIO generating and interrupt every time there is a falling edge. Moreover, enabling the bit-clearing option, any write operation to the edge capture register will clear its contents. Since this will be the port connected to the pushbuttons (*KEY*) it is useful to rename it: right click the PIO, select **`Rename`** and rename the port as *keys*. Rename also the *external connection* as *keys*.

Connect the *irq* port of the input PIO to the *platform_irq_rx* port of the Nios V processor.

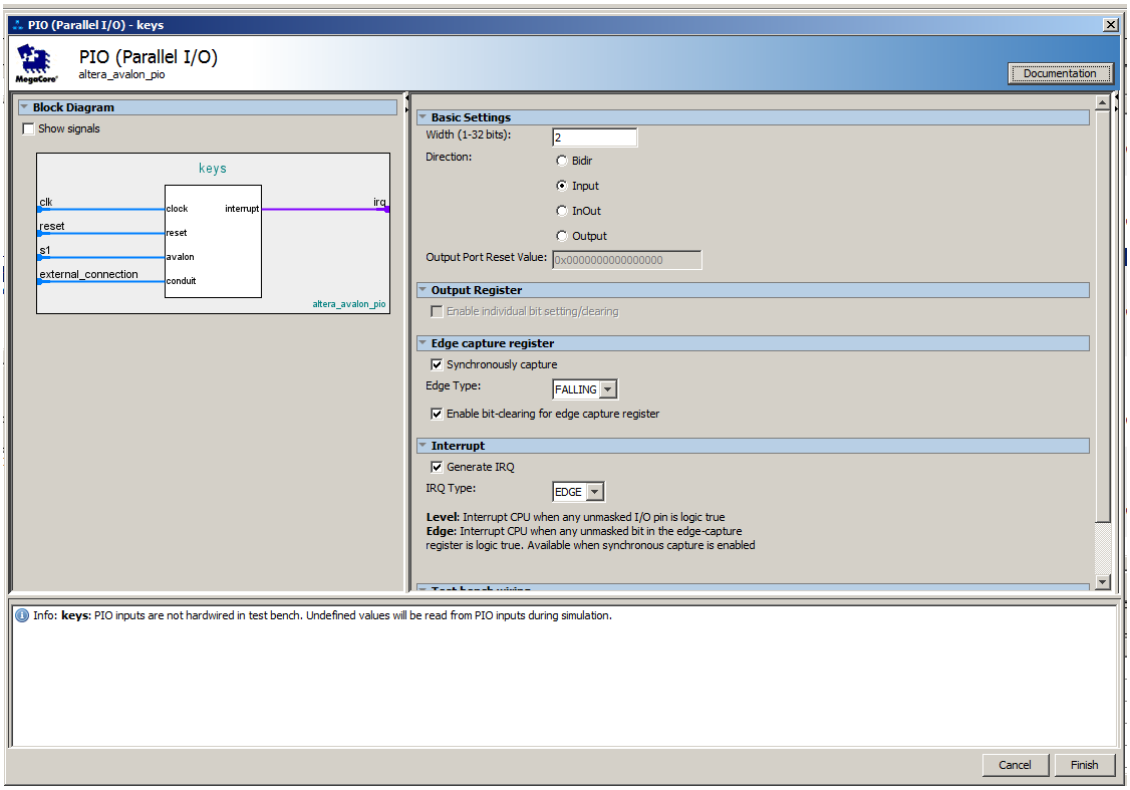


Figure 7 Input PIO interrupt generation

4. Adding a Timer

In order to insert a Timer, go to **`Processors and Peripherals > Peripherals > Interval Timer`** and click **`Add.`** The window, shown in Figure 8, appears: choose a counter size of 32 bits and the *Readable Snapshot* option. Then, click **`Finish.`**

Now you have to connect the timer to system:

- connect the timer clock port (*clk*) to the *sys_clk* (*clocks* component)
- connect the timer reset port (*reset*) to the *reset_source* (*clocks* component)
- connect the timer data port *(s1)* to the *data manager* (*Nios V2_QSYS (Platform Designer)_0* component)
- connect the timer interrupt port *(irq*) to the *platform_irq_rx* port of the Nios V processor (*Nios V2_QSYS (Platform Designer)_0* component)

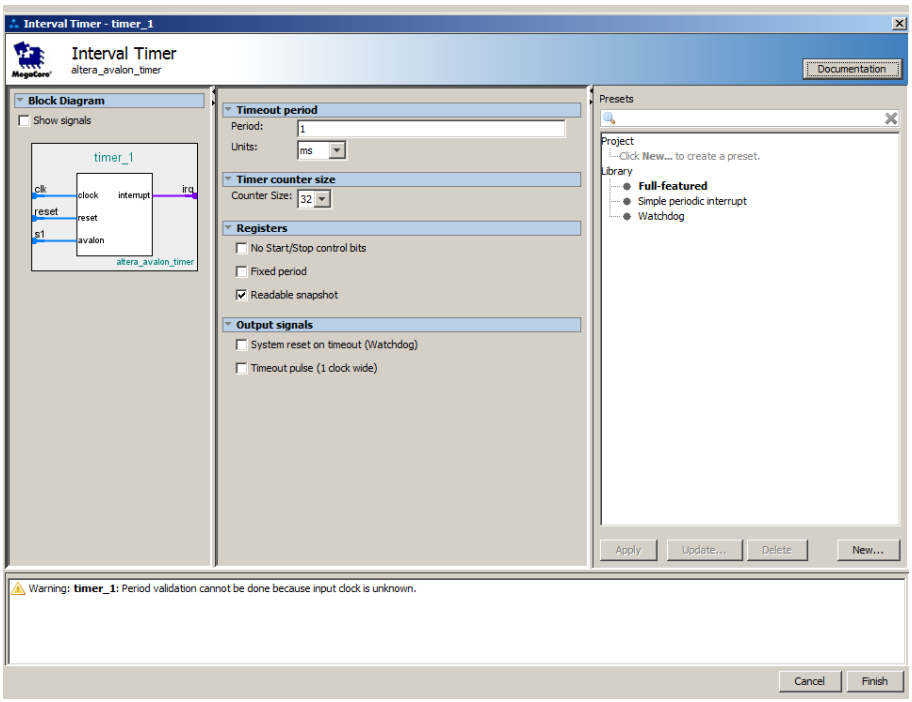


Figure 8 Interval Timer

5. Adding a Performance Counter

In order to insert a Performance Counter, go to **`Basic Functions > Simulation; Debug and Verification > Debug and Performance > Performance Counter Unit`** and click **`Add.`** The window, shown in Figure 9, appears, leave the default settings and click **`Finish.`**

Now you have to connect the performance counter to system:

- connect the clock port (*clk*) to the *sys_clk* (*clocks* component)
- connect the reset port (*reset*) to the *reset_source* (*clocks* component)
- connect the control port *(control slave)* to the *data manager* (*Nios V2_QSYS (Platform Designer)_0* component)

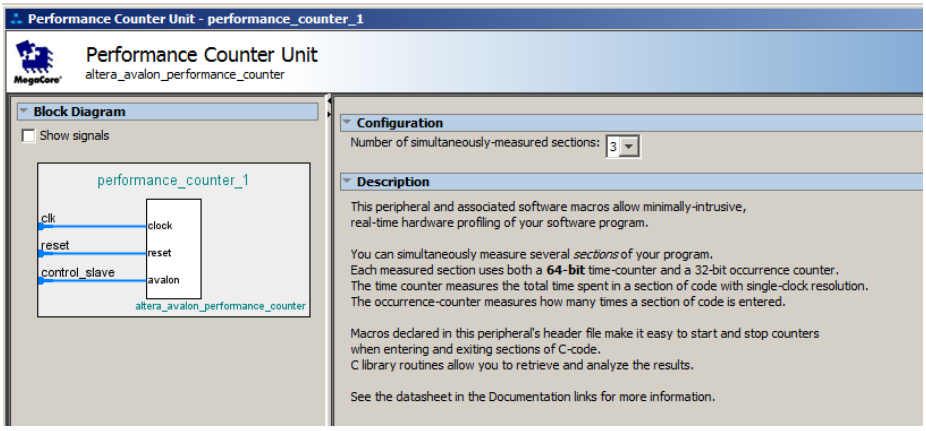


Figure 9 Performance counter

6. Building the final system

In order to build the final system you have to perform the following steps:

- check the on-chip-memory *Base* address is still locked at 0x00000000.
- click on **`System > Assign Base Addresses`** so the system is adjusting the base address of each peripheral (except the on-chip-memory).
- double-click on the *Base Address* of the SDRAM and enter the value 0x08000000 to produce the assignment shown in Figure 11.
- click on **`System > Assign Interrupt Numbers`** so the system is adjusting the IRQ number of each peripheral generating an interrupt.

To make use of the SDRAM, we need to configure the reset vector and exception vector of the Nios V processor. Right-click on the *Nios V_processor* and then select **`Edit`** to reach the window in Figure 10. Select *sdram.s1* to be the memory device for the *Reset Agent*, as shown in the figure. Click **`Finish`** to return to the System Contents tab.

The final system you obtain should look like the one shown in Figure 11 (a) and 11 (b). Now, the system is ready you can click on **`Generate HDL(*)`**.

When the HDL generation is complete you can start integrating the Nios V based system into your Quartus Prime project. The entity of the system generated by QSYS (Platform Designer) should look like the one shown in Figure 12.

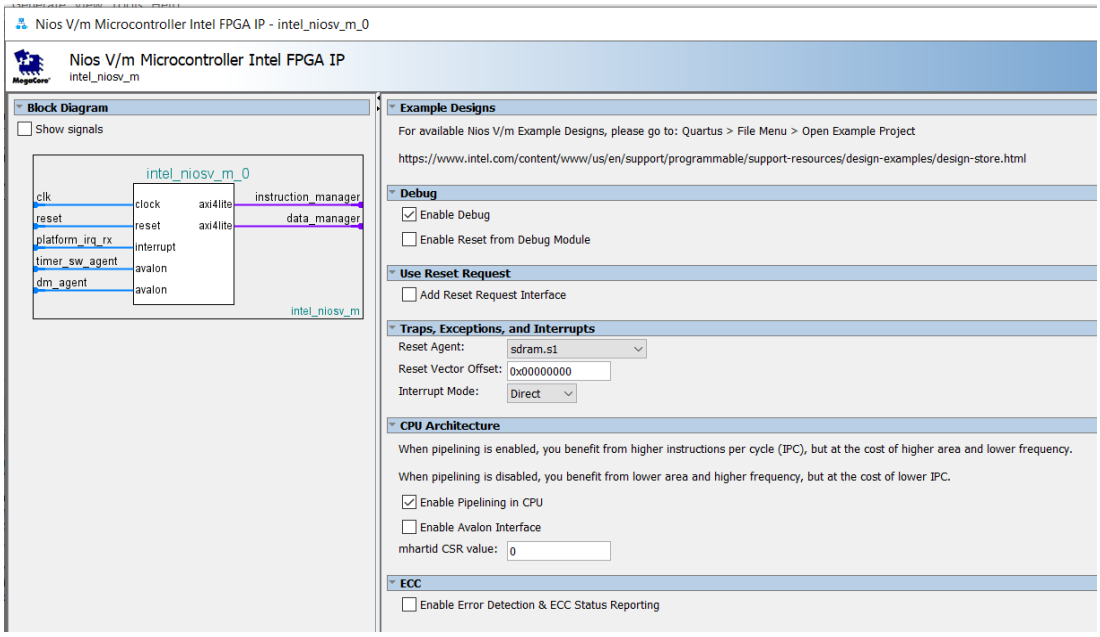


Figure 10 Define the Reset Agent.

(*) Note: if a copy of the *altera_avalon_new_sdram_controller* folder is not placed also in Quartus installation folder (<*quartus_path*>/*ip/altera/sopc_builder_ip*), the HDL generator will probably complain.

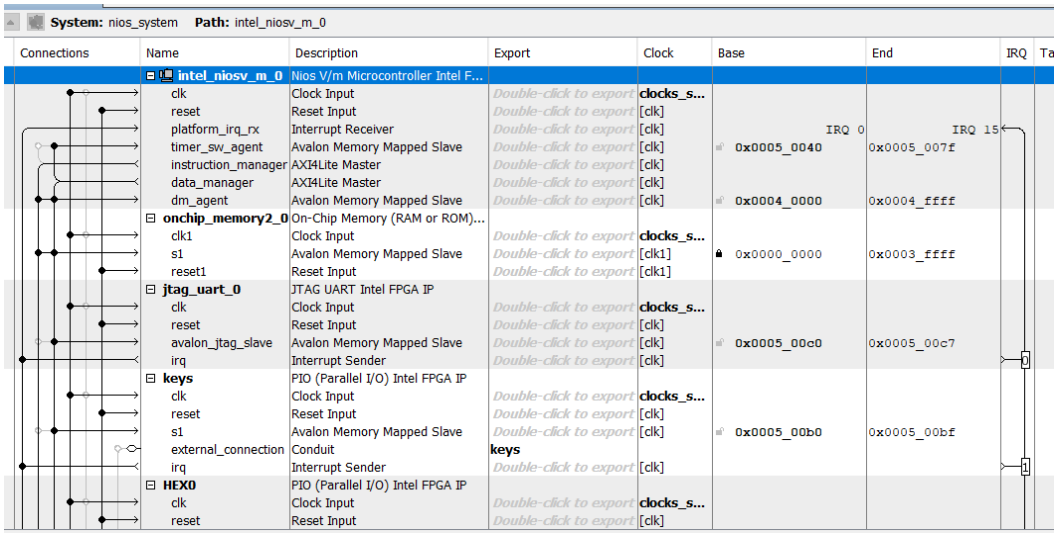


Figure 11 (a) Final system

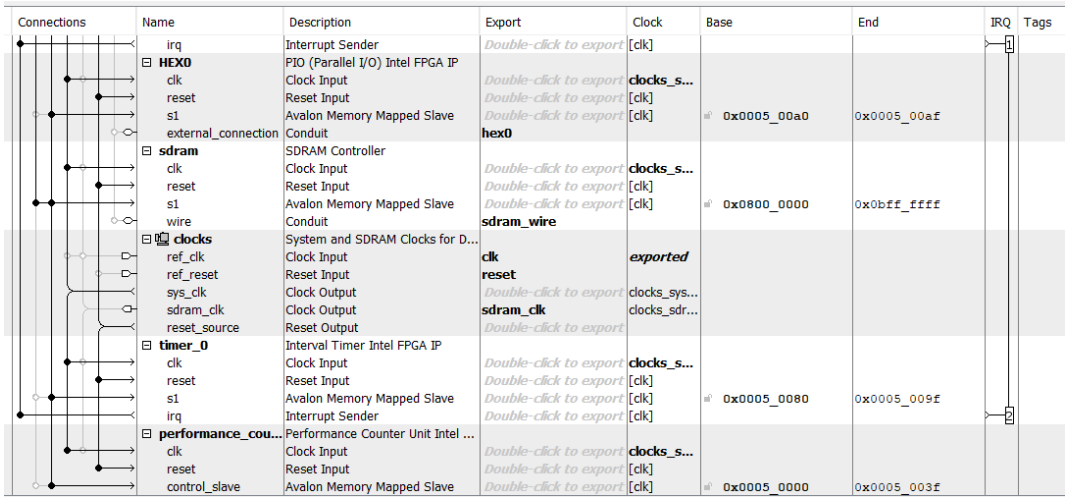


Figure 11 (b) Final system

```
// Verilog
module nios_system (
    output [6:0]  hex0_export,      // export
    input  [1:0]  keys_export,      // export
    output [12:0] sdram_wire_addr,  // addr
    output [1:0]  sdram_wire_ba,    // ba
    output        sdram_wire_cas_n, // cas_n
    output        sdram_wire_cke,   // cke
    output        sdram_wire_cs_n,  // cs_n
    inout  [15:0] sdram_wire_dq,    // dq
    output [1:0]  sdram_wire_dqm,   // dqm
    output        sdram_wire_ras_n, // ras_n
    output        sdram_wire_we_n,  // we_n
    output        sdram_clk_clk,    // clk
    input         clk_clk,          // clk
    input         reset_reset       // reset
);
Endmodule



-- VHDL
  entity nios_system is
    port (
      hex0_export      : out   std_logic_vector(6 downto 0);          -- export
      keys_export      : in    std_logic_vector(1 downto 0)  := (others => 'X');  -- export
      sdram_wire_addr  : out   std_logic_vector(12 downto 0);         -- addr
      sdram_wire_ba    : out   std_logic_vector(1 downto 0);          -- ba
      sdram_wire_cas_n : out   std_logic;                             -- cas_n
      sdram_wire_cke   : out   std_logic;                             -- cke
      sdram_wire_cs_n  : out   std_logic;                             -- cs_n
      sdram_wire_dq    : inout std_logic_vector(15 downto 0) := (others => 'X');  -- dq
      sdram_wire_dqm   : out   std_logic_vector(1 downto 0);          -- dqm
      sdram_wire_ras_n : out   std_logic;                             -- ras_n
      sdram_wire_we_n  : out   std_logic;                             -- we_n
      sdram_clk_clk    : out   std_logic;                             -- clk
      clk_clk          : in    std_logic                    := 'X';  -- clk
      reset_reset      : in    std_logic                    := 'X'   -- reset
      );
  end nios_system;
```

Figure 12 A part of the generated Verilog/VHDL

The *nios_system* module has to be instantiated in a top-level module like in the last lab. Figure 13 (a) and (b) show a top-level HDL module that instantiates the Nios V system (Verilog and VHDL). Now the system can be implemented with the procedure used in the last lab.

```
module lights (
    input  [2:0]  KEY,
    input         CLOCK_50,
    output [6:0]  HEX0,
    inout  [15:0] DRAM_DQ,
    output [12:0] DRAM_ADDR,
    output [1:0]  DRAM_BA,
    output        DRAM_CAS_N,
    output        DRAM_RAS_N,
    output        DRAM_CLK,
    output        DRAM_CKE,
    output        DRAM_CS_N,
    output        DRAM_WE_N,
    output        DRAM_UDQM,
    output        DRAM_LDQM
);

    // Internal wires to match the ports of nios_system
    wire [1:0] sdram_dqm;

    nios_system u_nios_system (
        .hex0_export      (HEX0),
        .keys_export      (KEY[2:1]),
        .sdram_wire_addr  (DRAM_ADDR),
        .sdram_wire_ba    (DRAM_BA),
        .sdram_wire_cas_n (DRAM_CAS_N),
        .sdram_wire_cke   (DRAM_CKE),
        .sdram_wire_cs_n  (DRAM_CS_N),
        .sdram_wire_dq    (DRAM_DQ),
        .sdram_wire_dqm   (sdram_dqm),
        .sdram_wire_ras_n (DRAM_RAS_N),
        .sdram_wire_we_n  (DRAM_WE_N),
        .sdram_clk_clk    (DRAM_CLK),
        .clk_clk          (CLOCK_50),
        .reset_reset      (~KEY[0]) // Active low reset
    );

    // Map individual bits of sdram_dqm to outputs
    assign DRAM_UDQM = sdram_dqm[1];
    assign DRAM_LDQM = sdram_dqm[0];

endmodule
```

Figure 13 (a) Instantiating the Nios V system using Verilog code

```
library ieee
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity lights is
  port (
    KEY                            : in    std_logic_vector(2 downto 0);
    CLOCK_50                       : in    std_logic;
    HEX0                           : out   std_logic_vector(6 downto 0);
    DRAM_DQ                        : inout std_logic_vector (15 downto 0);
    DRAM_ADDR                      : out   std_logic_vector (12 downto 0);
    DRAM_BA                        : out   std_logic_vector (1 downto 0);
    DRAM_CAS_N, DRAM_RAS_N, DRAM_CLK : out   std_logic;
    DRAM_CKE, DRAM_CS_N, DRAM_WE_N   : out   std_logic;
    DRAM_UDQM, DRAM_LDQM           : out   std_logic);
end lights;

architecture Structure of lights is

  component nios_system is
    port (
      hex0_export      : out   std_logic_vector(6 downto 0);          -- export
      keys_export      : in    std_logic_vector(1 downto 0)  := (others => 'X');  -- export
      sdram_wire_addr  : out   std_logic_vector(12 downto 0);         -- addr
      sdram_wire_ba    : out   std_logic_vector(1 downto 0);          -- ba
      sdram_wire_cas_n : out   std_logic;                             -- cas_n
      sdram_wire_cke   : out   std_logic;                             -- cke
      sdram_wire_cs_n  : out   std_logic;                             -- cs_n
      sdram_wire_dq    : inout std_logic_vector(15 downto 0) := (others => 'X');  -- dq
      sdram_wire_dqm   : out   std_logic_vector(1 downto 0);          -- dqm
      sdram_wire_ras_n : out   std_logic;                             -- ras_n
      sdram_wire_we_n  : out   std_logic;                             -- we_n
      sdram_clk_clk    : out   std_logic;                             -- clk
      clk_clk          : in    std_logic                    := 'X';  -- clk
      reset_reset      : in    std_logic                    := 'X'   -- reset
      );
  end component nios_system;

begin

  Nios_V : component nios_system
    port map (
      hex0_export       => HEX0,
      keys_export       => KEY(2 downto 1),
      sdram_wire_addr   => DRAM_ADDR,
      sdram_wire_ba     => DRAM_BA,
      sdram_wire_cas_n  => DRAM_CAS_N,
      sdram_wire_cke    => DRAM_CKE,
      sdram_wire_cs_n   => DRAM_CS_N,
      sdram_wire_dq     => DRAM_DQ,
      sdram_wire_dqm(1) => DRAM_UDQM,
      sdram_wire_dqm(0) => DRAM_LDQM,
      sdram_wire_ras_n  => DRAM_RAS_N,
      sdram_wire_we_n   => DRAM_WE_N,
      sdram_clk_clk     => DRAM_CLK,
      clk_clk           => CLOCK_50,
      reset_reset       => not KEY(0)
      );

end Structure;
```

Figure 13 (b) Instantiating the Nios V system using VHDL code

7. Developing the software

To develop the software application we can exploit the *Ashling RiscFree IDE for Intel FPGAs* as the last lab. Since there is no difference in setting up the project with respect to the last lab, we can concentrate on the C code.

Include files

Besides the *stdio.h* file, we need to include also all the file which define the parameters and the functions provided by our hardware (see Figure 14)

```
#include <stdio.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"
#include "altera_avalon_timer_regs.h"
#include "altera_avalon_performance_counter.h"
#include "sys/alt_irq.h"
```

Figure 14 Include files

Main program, initialization routine and performance monitor

The code of the main program is rather straightforward (see Figure 15). Note the insertion of the performance monitor statements. *PERF_RESET* and *PERF_START_MEASURING* are general command which reset and prepare all counters. *PERF_BEGIN (base, counter_number)* and *PERF_END(base, counter_number)* respectively start and stop the respective counter. Routine *perf_get_section_time(base,counter_number)* returns the value of the elapsed time as number of clock cycles. Since when entering the main, the CPU has the global interrupt register enabled, it is better to disable interrupts before initiating the peripherals and re-enable interrupts after the initialization. In this example, when the initialization is over we have a critical section where we want to measure the execution time of a computing task. Thus interrupts are re-enabled only after completing the measurement.

```
int main()
{
int i,j,k,a[13];
long time;
alt_irq_context context;

printf("Software demo\n");

context = alt_irq_disable_all();

init_all();
flag = 0;
PERF_RESET(PERFORMANCE_COUNTER_0_BASE);
PERF_START_MEASURING(PERFORMANCE_COUNTER_0_BASE);
PERF_BEGIN(PERFORMANCE_COUNTER_0_BASE,1);
/* COMPUTING TASK */
for (k=0; k<5001; k++)
{
       j=0;
       for (i=0; i<13; i++)
       {
              if (i==0)
                     j =1;
              else
                     j=j*i;
```

```
            a[i]=j;
      }
}
PERF_END(PERFORMANCE_COUNTER_0_BASE,1);
PERF_STOP_MEASURING(PERFORMANCE_COUNTER_0_BASE);

alt_irq_enable_all(context);

printf("%d %d\n",i,a[12]);
time=(long)perf_get_section_time((int*)PERFORMANCE_COUNTER_0_BASE,1);
time= time/50;
printf("Time = %ld usec\n",time);

while (1);
}
```

Figure 15 the main program code

The initialization routine, called from the main performs all initialization tasks as shown in Figure 16

```
/* INITIALIZATION ROUTINE */
void init_all()
{
      init_timer();
      init_BUTTONS_interrupts();
      global_cnt =0;
      prescaler =0;
      key_edge=0;
}
```

Figure 16 the initialization routine

Timer initialization and interrupt service routine

Figures 17 shows the timer interrupt and initialization routines. Timer is initialized to generate an interrupt every fifth of a second. The interrupt service routine updates the display every interrupt or every 5 occurrences, depending on the value of global variable *key_edge*. The *display* function is shown in Figure 18.

```
/*
* TIMER INTERRUPT ROUTINE
*/
static void timer_isr(void *context)
{
      /* Clear the interrupt */
      IOWR_ALTERA_AVALON_TIMER_STATUS(TIMER_0_BASE, 0x00);
      if ((key_edge&0x03)==0)
      {
            global_cnt=0;
      }
      else
      {
            if((key_edge&0x03)==1)
            {
                  global_cnt +=1;
            }
            else
```

```
		{
			prescaler+=1;
			if (prescaler == 5)
			{
				prescaler =0;
				global_cnt+=1;
			}
		}
	}
	display(global_cnt);
}
/*
* TIMER INITIALIZATION ROUTINE
*/
void init_timer()
{
/* Force counter stop */
	IOWR_ALTERA_AVALON_TIMER_CONTROL(TIMER_0_BASE,
ALTERA_AVALON_TIMER_CONTROL_STOP_MSK);
/* Clear all flags in status register */
	IOWR_ALTERA_AVALON_TIMER_STATUS(TIMER_0_BASE, 0x00);
/* Write in the PERIOD register the value 0x989680 corresponding to 10.000.000 */
	IOWR_ALTERA_AVALON_TIMER_PERIODL(TIMER_0_BASE, 0x9680);
	IOWR_ALTERA_AVALON_TIMER_PERIODH(TIMER_0_BASE, 0x0098);
	test=alt_ic_isr_register(TIMER_0_IRQ_INTERRUPT_CONTROLLER_ID, TIMER_0_IRQ,
timer_isr, NULL,NULL);
	if (test == 0)
		printf("Timer Interrupt Routine Registered\n");
	IOWR_ALTERA_AVALON_TIMER_CONTROL(TIMER_0_BASE, ALTERA_AVALON_TIMER_CONTROL_ITO_MSK
| ALTERA_AVALON_TIMER_CONTROL_CONT_MSK | ALTERA_AVALON_TIMER_CONTROL_START_MSK);
}
```

Figure 17 Timer routines

```
/*
* DISPLAY ROUTINE
*/
unsigned char table[16]={0x40, 0x79, 0x24, 0x30, 0x19, 0x12, 0x02, 0x78, 0x00, 0x10, 0x08,
0x03, 0x46, 0x21, 0x06, 0x0e};

void display(unsigned char bcd)
{
	IOWR_ALTERA_AVALON_PIO_DATA(HEX0_BASE, table[bcd&0x0f]&0x7f);
}
```

Figure 18 The *display* function

The button initialization and the interrupt routine

The button initialization and interrupt service routine are shown in Figure 19. The interrupt routine updates the value of the *key_edge* global variable with the value of which push button has been last pressed.

```
/************************************
* BUTTONS Interrupt Service routine
*
*/
```

```
static void button_isr(void* context)
{
/* Read the edge capture register from the button PIO
into destination pointed by an appropriately cast pointer
*/
	*(volatile int*) context = IORD_ALTERA_AVALON_PIO_EDGE_CAP(KEYS_BASE);
	/* Write to the edge capture register to reset it */
	IOWR_ALTERA_AVALON_PIO_EDGE_CAP(KEYS_BASE, 0x3);
}
/*
* BUTTONS Initialization Routine
*/
void init_BUTTONS_interrupts()
{
	/* Reset the edge capture register. */
	IOWR_ALTERA_AVALON_PIO_EDGE_CAP(KEYS_BASE, 0x3);
	/*
	 * Register the ISR.
	 * Uses the new API interface
	 *
	 * */
	test=alt_ic_isr_register(KEYS_IRQ_INTERRUPT_CONTROLLER_ID, KEYS_IRQ, button_isr,
(void*) &key_edge, 0x0);
	if (test == 0)
		printf("PIO Interrupt Routine Registered\n");
	/* Enable 2 button interrupts */
	IOWR_ALTERA_AVALON_PIO_IRQ_MASK(KEYS_BASE, 0x3);
}
```

Figure 19 Button routines

Part of this tutorial is derived from Intel-Altera document "*Using the SDRAM on Intel's DE1-SoC Board with VHDL Designs*".

Part of this tutorial is derived from the material prepared by Prof. F. Gregoretti, DET, Politecnico di Torino.